

Лабораторна робота №1

Розв'язання задачі класифікації за допомогою одного нейрона (одношарового перцептрону) мовою Python

Мета роботи:

Отримати навички підготовки даних, побудови та навчання найпростішої ШНМ для вирішення завдань класифікації, а також її програмування мовою Python. Навчитися проводити експерименти з ШНМ, ознайомитись з деякими функціями бібліотек Python.

Завдання:

1. Встановіть Python 3 або вище (завантажте Python за посиланням <https://www.python.org/downloads/>).

Варіанти для роботи з Python:

- завантажити кросплатформове середовище розробки PyCharm за посиланням <https://www.jetbrains.com/pycharm/download/#section=windows>, або
- відкрити безкоштовне хмарне середовище для роботи з Python - блокнот Colab у браузері за посиланням <https://colab.research.google.com/drive/>).

Вам також знадобляться бібліотеки pandas та numpy.

2. Підготуйте навчальні дані за допомогою Excel.

Як приклад, розглянемо підготовку набору даних (навчальну вибірку) для навчання одношарового перцептрона з допомогою таблиць Excel (рис. 1). Створимо навчальну вибірку із 500 прикладів. Нехай x , y , z – входи нейрона (стовпці A, B, C), output (стовпець D) – потрібна реакція нейрона.

	D2			f_x	=IF(A2<0, 0, 1)
	A	B	C	D	E
1	x	y	z	output	
2	-3	8	-10	0	
3	-1	-6	-1	0	
4	-9	-2	-5	0	
5	-10	6	-5	0	
6	0	-2	7	1	
7	6	-5	-1	1	
8	9	-2	3	1	
9	3	10	-7	1	
10	5	2	4	1	
11	8	-7	-6	1	
12	-2	2	-9	0	
13	-5	7	9	0	
14	3	-6	-2	1	
15	-8	-9	8	0	
16	-2	9	-9	0	
17	10	-2	3	1	
18	1	-2	7	1	

Рис. 1.

У цій таблиці для формування значень x, y, z використовувалася функція RANDBETWEEN (СЛУЧМІЖ), яка повертає випадкове ціле число між двома встановленими межами (у даному випадку – між 10 та -10). Для формування значень output використовується формула IF (ЯКЩО), вид якої наведено на рис.1. Навчальна вибірка може (і має) використовуватися кілька разів. Її повне одноразове використання (повний прохід) називається однією епохою.

3. Створіть архітектуру одношарового перцептронну.

Приклад архітектури подано на рис.2.

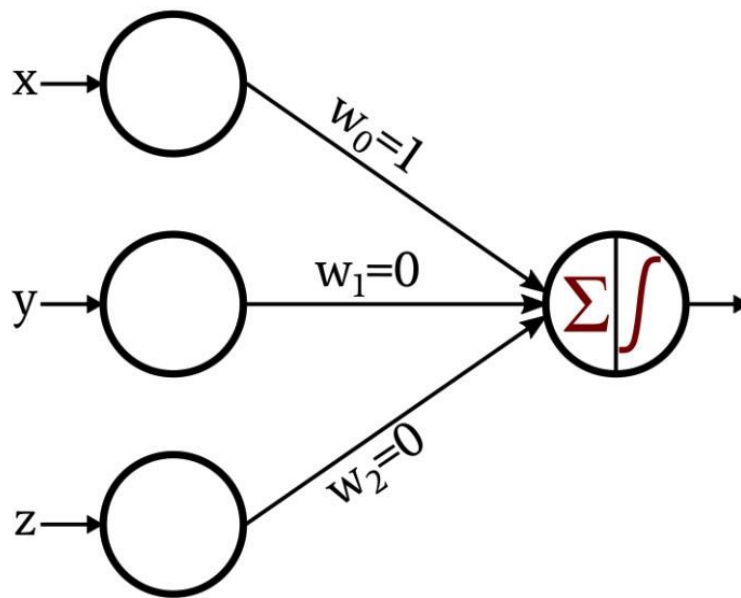


Рис.2.

Нехай, вагові коефіцієнти $w_0=1$, w_1 та w_2 дорівнюють 0 (що у цьому випадку не важливо). При ініціалізації ці вагові коефіцієнти мають отримати випадкові значення діапазоні $[0, 1]$.

Функція активації у вихідному вузлі – одинична ступінчаста функція (функція Хевісайда). Значення функції дорівнює 0 при зваженій сумі входів (потенціалі активації) < 0 та 1 при зваженій сумі входів ≥ 0 .

4. Розробіть алгоритм побудови та навчання одношарового перцептрону, напишіть код.

Як приклад, напишемо код для побудови та навчання одношарового перцептрону та пояснимо його. Код наведено нижче.

```
import pandas
import numpy as np

input_dim = 3

learning_rate = 0.01

Weights = np.random.rand(input_dim)
```

```

#Weights[0] = 0.5
#Weights[1] = 0.5
#Weights[2] = 0.5

Training_Data = pandas.read_excel("3D_data.xlsx")

Expected_Output = Training_Data.output

Training_Data = Training_Data.drop(['output'], axis=1)

Training_Data = np.asarray(Training_Data)

training_count = len(Training_Data[:,0])

for epoch in range(0,5):

    for datum in range(0, training_count):

        Output_Sum = np.sum(np.multiply(Training_Data[datum,:],
Weights))

        if Output_Sum < 0:
            Output_Value = 0
        else:
            Output_Value = 1

        error = Expected_Output[datum] - Output_Value

        for n in range(0, input_dim):
            Weights[n] = Weights[n] +
learning_rate*error*Training_Data[datum,n]

print("w_0 = %.3f" %(Weights[0]))

print("w_1 = %.3f" %(Weights[1]))

print("w_2 = %.3f" %(Weights[2]))

```

5. Пояснення коду.

5.1. Налаштування мережі та організація даних.

Імпортуємо бібліотеки pandas та numpy.

```

import pandas
import numpy as np

```

Задаємо розмірність мережі (кількість вхідних вузлів).

```
input_dim = 3
```

Розмірність вхідних даних регулюється. Наші вхідні дані складаються з тривимірних координат, тому нам потрібні три вхідні вузли. Ця програма не підтримує декілька вихідних вузлів.

Задаємо швидкість навчання.

```
learning_rate = 0.01
```

Нехай швидкість навчання дорівнюватиме 0,01 (значення швидкості навчання розглянемо в наступній роботі).

Встановимо початкові значення вагових коефіцієнтів випадковим чином.

```
Weights = np.random.rand(input_dim)
```

Функція бібліотеки numpy `random.rand()` генерує масив довжиною `input_dim`, заповнений випадковими значеннями, розподіленими в інтервалі `[0, 1]`. Початкові значення ваги впливають на їх кінцеві значення, отримані в результаті процедури навчання. Тому, якщо ви хочете оцінити вплив інших гіперпараметрів (таких як розмір навчального набору або швидкість навчання) на процес навчання, ви можете встановити всі ваги в значення відомої константи замість випадково згенерованого числа, наприклад, так, як представлено в коментарях коду.

```
#Weights[0] = 0.5  
#Weights[1] = 0.5  
#Weights[2] = 0.5
```

Використовуємо бібліотеку pandas для імпорту навчальних даних із електронної таблиці Excel.

```
Training_Data = pandas.read_excel("3D_data.xlsx")
```

Набір навчальних даних включає в себе вхідні значення та відповідні вихідні значення. Перша інструкція нижче виділяє вихідні значення і зберігає їх у окремому масиві, а наступна інструкція видаляє вихідні значення з початкового набору навчальних даних.

```
Expected_Output = Training_Data.output
```

```
Training_Data = Training_Data.drop(['output'], axis=1)
```

Перетворимо набір навчальних даних, який в даний час є структурою даних pandas, у масив numpy, а потім переглянемо довжину одного зі стовпців, щоб визначити, скільки точок даних доступно для навчання.

```
Training_Data = np.asarray(Training_Data)
```

```
training_count = len(Training_Data[:,0])
```

5.2. Розрахунок вихідних значень.

Встановимо кількість епох навчання. Тривалість одного тренування визначається кількістю доступних навчальних даних (прикладів, що містяться у навчальній вибірці). Тим не менш, з метою мінімізації помилки необхідно продовжити навчання, повторюючи подачу на входи мережі прикладів з

навчальної вибірки. Кожен повний прохід через весь навчальний набір називається епохою.

```
for epoch in range(0,5):
```

Далі, створимо процедуру, яка міститься в циклі і виконується один раз для кожного рядка в навчальному наборі, де "рядок" відноситься до групи значень вхідних даних і відповідного вихідного значення (у нашому випадку вхідна група складається з трьох чисел, що представляють компоненти x , y та z точки у тривимірному просторі).

```
for datum in range(0, training_count):
```

Створимо реакцію вихідного вузла. Вихідний вузол повинен підсумовувати значення, одержані трьома вхідними вузлами. Для цього спочатку виконаємо поелементне множення масиву `Training_Data` на масив `Weights`, а потім обчислимо суму елементів у масиві, отриманому цим множенням.

```
Output_Sum = np.sum(np.multiply(Training_Data[datum,:], Weights))
```

Далі, застосуємо оператор `if-else`, який застосовує одиничну ступінчасту функцію активації без урахування значення порога (для спрощення): якщо сума менша за нуль, то значення, згенероване вихідним вузлом, дорівнює 0; якщо сума дорівнює або більше нуля - вихідне значення дорівнює одиниці.

```
if Output_Sum < 0:
    Output_Value = 0
else:
    Output_Value = 1
```

5.3. Оновлення ваги.

Спочатку згадаємо трохи теорії. Коли перший розрахунок вихідних значень завершено, ми маємо вагові значення, згенеровані випадковим чином. Але ці значення не дозволяють нам досягти правильної класифікації. Ми перетворюємо нейронну мережу на ефективну систему класифікації, багаторазово змінюючи ваги таким чином, щоб вони поступово відображали математичне співвідношення між вхідними даними та заданими вихідними значеннями. Зміна ваги досягається шляхом застосування наступного правила навчання для кожного рядка в навчальному наборі:

$$w_{\text{новий}} = w + (\alpha \times (\text{вихід}_{\text{очікуваний}} - \text{вихід}_{\text{розрахований}}) \times \text{вхід})$$

Символ α означає швидкість навчання. Таким чином, щоб обчислити нове значення ваги, ми множимо відповідне вхідне значення на швидкість навчання і на різницю між очікуваним вихідним значенням (яке забезпечується навчальним набором) та розрахованим вихідним значенням, а потім результат цього множення додається до поточного значення ваги. Якщо ми визначимо дельту (δ) як $(\text{вихід}_{\text{очікуваний}} - \text{вихід}_{\text{розрахований}})$, ми можемо переписати формулу як

$$w_{\text{новий}} = w + (\alpha \times \delta \times \text{вхід})$$

Тоді код процедури оновлення ваги (навчання) буде мати такий вигляд.

```
error = Expected_Output[datum] - Output_Value

for n in range(0, input_dim):
    Weights[n] = Weights[n] +
learning_rate*error*Training_Data[datum,n]
```


6. Індивідуальні завдання:

1. Виберіть архітектуру мережі (кількість вхідних вузлів) і *створіть навчальний набір даних* за допомогою Excel, використовуючи **свою функцію формування значень output**. Нормалізуйте вхідні значення в інтервалі $[0, 1]$. *Створіть тестовий набір даних* або тестову вибірку. Ці дані не братимуть участі у навчанні. Примітка: використовуйте прості функції для формування бінарних значень output. Ще один приклад, який надається виключно для ілюстрації і який **НЕ ТРЕБА** використовувати у якості навчальної вибірки, наведено на рис. 3.

	Input			Output
Example 1	0	0	1	0
Example 2	1	1	1	1
Example 3	1	0	1	1
Example 4	0	1	1	0
New situation	1	0	0	?

Рис.3.

Зверніть увагу, що в даному прикладі значення стовпця Output завжди дорівнює значенню самої лівої колонки зі стовпця Input. Це означає, що правильна відповідь у нашому випадку дорівнюватиме 1.

2. Використовуйте у своїй мережі сигмоїдну активаційну функцію, а не ступінчасту, як було у розглянутому прикладі.

3. Створіть процедуру навчання мережі. Для модифікації вагових коефіцієнтів використовуйте значення зваженої помилки (1):

$$\Delta W = error \times input \times Grad(output) \quad (1)$$

Формула (1) дозволяє виконувати коригування ваги пропорційно до величини помилки (*error*). Далі множення відбувається на вхідне значення *input*. Якщо вхідне значення дорівнює 0, вага коригуватися не буде. Потім виконується множення на *градієнт* сигмоїдної кривої (*Grad (output)*).

Потрібно врахувати, що при великих числах крива має невеликий градієнт. Спрощено процес модифікації вагових коефіцієнтів можна описати так: якщо нейрон "впевнений" у правильності існуючої ваги, він "не хоче" коригувати його занадто сильно. Множенням на градієнт кривої саме це й досягається. Для обчислення градієнта потрібно взяти похідну. У спрощеному вигляді це буде (2):

$$Grad(output) = output \times (1 - output) \quad (2)$$

Якщо ми відніmemo 2-е рівняння з 1-го, то отримаємо необхідну підсумкову формулу:

$$\Delta W = error \times input \times output \times (1 - output)$$

Вам також можуть знадобитися деякі додаткові функції з бібліотеки *numpy*, наприклад,

- *exp* - для побудови експоненти;
- *dot* - для перемноження матриць;
- *array* - для створення матриці;
- *random* - для генерування випадкових чисел та ін.

4. Навчіть мережу, використовуючи 10, 50, 100, 1000 епох. Отримайте мінімальну помилку перцептрону на навчальному сеті та підсумкові вагові коефіцієнти.

5. Побудуйте алгоритм тестування та проведіть тестування перцептрону на перевірконому сеті.

6. Зробіть висновки з огляду на аналіз значень гіперпараметрів.

7. Зміст звіту:

- архітектура мережі, вибрані гіперпараметри ШНМ;
- алгоритм побудови навчальної та тестової вибірки;
- алгоритм навчання ШНМ;
- текст програми з коментарями;
- результат навчання (скріншот);
- висновки.